

Getting pwn'd by AI: Penetration Testing with Large Language Models

Andreas Happe

andreas.happe@tuwien.ac.at
TU Wien
Austria

Jürgen Cito

juergen.cito@tuwien.ac.at
TU Wien
Austria

ABSTRACT

The field of software security testing, more specifically penetration testing, requires high levels of expertise and involves many manual testing and analysis steps. This paper explores the potential use of large-language models, such as GPT3.5, to augment penetration testers with AI sparring partners. We explore two distinct use cases: high-level task planning for security testing assignments and low-level vulnerability hunting within a vulnerable virtual machine. For the latter, we implemented a closed-feedback loop between LLM-generated low-level actions with a vulnerable virtual machine (connected through SSH) and allowed the LLM to analyze the machine state for vulnerabilities and suggest concrete attack vectors which were automatically executed within the virtual machine. We discuss promising initial results, detail avenues for improvement, and close deliberating on the ethics of AI sparring partners.

CCS CONCEPTS

• Security and privacy → Systems security.

KEYWORDS

security testing, penetration testing, large language models

ACM Reference Format:

Andreas Happe and Jürgen Cito. 2023. Getting pwn'd by AI: Penetration Testing with Large Language Models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*, December 3–9, 2023, San Francisco, CA, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3611643.3613083>

1 INTRODUCTION

Large language models (LLMs), such as ChatGPT or GPT3.5, have become a hot topic not only in computer science but also within popular media [12]. The field of cybersecurity and software security testing, more specifically, penetration testing, suffers from a chronic lack of personnel [19], even worse, according to the ISC2 Cybersecurity Workforce Study 2022 [18], while global cybersecurity workforce was growing by 11.1% YoY, this growth was outpaced by the gap's increase of 26.2% YoY. A recent interview study with penetration testers highlighted the need for human sparring partners [16], i.e., colleagues who offer alternative ideas or approaches

when stuck. The study also emphasizes that intuition is a big part of detecting vulnerabilities and that knowledge transfer, e.g., from attending Capture-the-Flag¹ (CTF) events, were seen as potential sources of this intuition — can this be partially outsourced to AI models? Using AI-based agents as sparring partners would augment and empower existing human security testers and could counteract the lack of sufficiently educated security professionals. Combining human operators with AIs creates new capabilities instead of cloning existing ones. Furthermore, keeping a human in the loop reduces the potential ethical problems imposed by the use of AIs [6]. Recent research indicates that the efficiency gains provided by the use of AI-based systems are greatest for low-skilled workers [7], augmenting human operators with a generative AI might thus also benefit the training of novice penetration testers.

RQ: To what extent can we automate security testing with LLMs? The rest of this paper explores whether large-language models can be deployed as sparring partners for security professionals. To answer this question, we leverage MITRE ATT&CK, a curated database of knowledge about threat actors in the cybersecurity domain, to provide a guiding structure. A good sparring partner should be able to cover the different tactics, techniques, and procedures covered by ATT&CK. To explore this hypothesis, we performed multiple experiments. To showcase high-level guidance, we “asked” an LLM to help design penetration tests for both generic scenarios as well as for a concrete target organization. To showcase low-level guidance, we integrated GPT3.5 with a vulnerable virtual machine and allowed it to analyze the machine for vulnerabilities and suggest attack vectors. Based on our experience, we discuss the results as well as potential future improvements.

Scope. We also envision other areas where generative AI could be used successfully. One of them is the generation of phishing or vishing messages. For obvious ethical reasons, we did not further analyze attacks that intently try to deceive human beings. Another tedious area where generative AI could improve efficiency would be automated report generation for penetration tests or red teaming campaigns. Anecdotal evidence suggests that penetration testers are already experimenting with generative AI for report generation.

2 BACKGROUND

This section highlights the technologies and techniques used.

MITRE ATT&CK. MITRE ATT&CK [37] is a curated database of knowledge about threat actors (APTs). It employs a hierarchical model often abbreviated by “TTP”. The initial “T” stands for “tactics” and describes high-level objectives an adversary intends to achieve,

¹CTFs are gamified penetration-testing exercises.

ESEC/FSE '23, December 3–9, 2023, San Francisco, CA, USA

© 2023 Copyright held by the owner/author(s).

This is the author's version of the work. It is posted here for your personal use. Not for redistribution. The definitive Version of Record was published in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*, December 3–9, 2023, San Francisco, CA, USA, <https://doi.org/10.1145/3611643.3613083>.

e.g., reconnaissance, privilege escalation or collection. The middle “T” describes “techniques”. Each technique is a way to achieve a tactic. Examples of techniques would be “*Abuse Elevation Control Mechanism: Sudo and Sudo Caching*” [3] or “*Steal or Forge Kerberos Tickets: Kerberoasting*” [4]. Finally, “P” describes procedures that are the specific details of how an adversary executes a technique.

We assume that a sparring partner for penetration testing should cover the whole TTP spectrum. On a high level, it should be able to select suitable tactics and corresponding techniques. On a low-level, given an employed tactic, it should be able to derive feasible techniques and procedures.

Large Language Models. A Large-Language Model (LLM) consists of a neural network trained using self-supervised learning on vast amounts of data. A model’s capabilities are highly dependent upon its complexity which is often described through the number of used parameters. Current models yield parameter sizes ranging from billions, e.g., LLaMA starts with 7 billion, to trillions of parameters, e.g., Wu Dao or GPT-4. Model and parameter sizes are currently under discussion; on one hand, larger models can exhibit emergent behaviors [38]; on the other hand, e.g., there is speculation that the age of ever-larger models is over due to reduced scaling efficiency [24].

Training a new LLM is prohibitively expensive for most researchers, but existing LLMs can be refined or fine-tuned to specific use cases for feasible costs. This situation has created the moniker “foundation models” for LLMs. The importance of those has been acknowledged by mainstream media, c.f., the Economist’s “Huge Foundation Models are Turbo Charging AI Progress” in 2022 [11].

GPT3.5/ChatGPT. Conversations with ChatGPT commonly consist of questions, named “prompts”, and answers going back and forth between the user and the AI. Prompts have to be carefully prepared, yielding a new discipline that has been called prompt engineering [10, 22, 36, 40].

Tools such as *llama.cpp* [14] that make use of small-scale models (up to 13b parameters) feasible on consumer-grade hardware have sparked additional research. Those models can be run without any cloud/API costs and are not subject to any server-side moderation or censorship.

Pre-trained Autonomous AI Agents. AutoGPT [15] introduced the idea of auto-generating sequences of instructions by leveraging LLMs to create the prompt that is subsequently used to query the LLM. This allows users to provide concise initial questions for the AI system that are subsequently refined. This reduces the need for manual prompt engineering. LLMs often “hallucinate”, i.e., invent facts that seem statistically plausible. Research suggests that using external knowledge and automated feedback can reduce these hallucinations [29]. AutoGPT integrates web-based queries and optional human-provided feedback during its operation. Based on this, the initial task is converted into a task list containing smaller subtasks that can be delegated to additional agents.

BabyAGI focuses on automated task generation, planning, and execution [26, 27]: a user-given task is split up into smaller subtasks that are stored within a task queue. *Autonomous Task Execution Agents* take tasks from the task queue, execute them, and add new information to a memory store. In addition, the *Task Creation Agent*

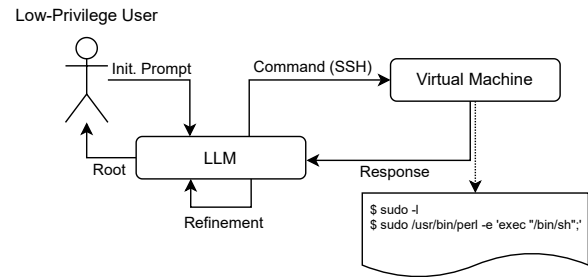


Figure 1: High-Level Architecture Overview

identifies new subsequent tasks that are pushed upon the task queue and are eventually executed by the *Task Execution Agent*. Before a task is executed, a *Context Agent* is asked to provide sufficient context for the task from memory. Entries in the task queue are prioritized through a *Prioritization Agent*. All mentioned agents are GPT-4 processes themselves. BabyAGI [25] provides a “pared-down” version of this system in roughly 100 lines of Python code. Jarvis [33] employs agents with different models to create multi-modal, multiagent systems.

3 LLM-BASED PENETRATION TESTING

We differentiate between two use cases: on a high level, typical questions asked by pen-testers are “what is a good attack methodology”, e.g., “how to attack Active Directory”. These questions should yield tactics as well as potential techniques to achieve those tactics. On the low-level, we assume that the pen-tester has already chosen to employ a tactic against a target system and is searching for appropriate techniques and corresponding procedures. A typical question would be “I want to perform a privilege escalation (tactic), what are the suitable attack vectors against [this concrete Linux system]?”.

3.1 High-Level: Task-Planning Systems

For the evaluation, we asked AgentGPT to “Become domain admin in an Active Directory”. The generated document contained highly realistic attack vectors such as password spraying, Kerberoasting, AS-REP roasting, exploiting Active Directory Certificate Services, abusing unconstrained delegation or exploiting group policies. All of those attacks are realistic, feasible, and commonly used during penetration testing.

In addition, after securing a target company’s approval, we tasked AutoGPT to devise an external penetration testing plan for that company. AutoGPT’s plan included standard methods such as performing a network vulnerability scan, performing OSINT/user enumeration, and performing phishing against identified users. All these are operations typically performed during external penetration tests. When further inquired, AutoGPT was able to crawl the company’s web page and identify potential phishing targets (users and their email addresses) but declined to perform any “real” network security scan or perform phishing operations due to its ethical filters. Both answers were realistic, and feasible, and would give a penetration tester good feedback about potential attack vectors.